

TraceFlow: Guiding Frozen Flow-Matching Robot Policies with Success and Failure Traces

Jiaxuan Zhang^{1,2}, Ruizhe Liu¹, Yu Zhang¹, and Yanchao Yang^{1,*}

Abstract—A vision-language-action (VLA) policy with a flow-matching action expert generates each action chunk (a short command sequence) by integrating a learned velocity field; once its weights are fixed, the success or failure of an earlier rollout cannot change the chunk generated now. Concurrent test-time methods give a frozen policy such an input from retrieved successes, a learned critic, a verifier, or a dynamics model, but none uses the robot’s own failed rollouts as negative evidence with nothing but a terminal outcome bit. We introduce TraceFlow, a progress-aligned guidance field that turns the action densities of retrieved successful and failed rollouts into a bounded correction to a frozen flow-matching action expert, using one terminal outcome bit per rollout and no other label. Its TraceBank stores traces, time-ordered state–action records with a terminal label, starts from the target-task training traces, and later admits the deployed robot’s own rollouts. On an ordered real-robot packing task the base completes 21 of 50 trials in order, TraceFlow 39, and one stacking round without any weight update 47, with wrong-sequence episodes falling from 20 to 0. In simulation the gain is selective: with per-suite selected settings, TraceFlow raises RoboMemArena Sequence from 78.92% to 91.50% task success and Transferring from 54.41% to 62.00% at stacking round 2, leaves the 26-task aggregate unchanged, lowers Counting and Occlusion by 1.12 and 1.42 points, and changes LIBERO-Plus (Long) by +1.27 points ($p = 0.0733$). Stacking gains are finite, every branch peaking before round ten, and the bank’s success-to-failure ratio predicts no retrieval allocation.

I. INTRODUCTION

Vision-language-action (VLA) policies pair a vision-language model (VLM) with an action expert that generates an action chunk from Gaussian noise; with flow matching, the expert integrates a learned velocity field over a fixed number of solver steps [1]–[7]. Once deployed with its parameters frozen (fixed for the whole deployment), the policy has no input through which the success or failure of an earlier rollout (one task attempt) can change the action chunk generated now. Several concurrent methods give a frozen policy such an input: Retrieve-then-Steer and OptimusVLA retrieve successful experience and inject a prior into the flow sampler [8], [9]; Guided Action Flow and test-time Q-guidance differentiate a learned critic [10], [11]; TACO and RoboMonkey sample several chunks and keep the one a learned verifier prefers [12], [13]; DynaGuide steers with a learned dynamics model [14]; and CCDP steers a diffusion policy away from the failed attempts of the current episode [15]. None of them uses the robot’s own failed rollouts,

labeled by nothing but a terminal bit, as repulsion at the current solver state next to successful rollouts as attraction, with no learned critic, verifier, or dynamics model and with an explicit bound on the correction; that is the gap this paper fills.

Long-horizon manipulation makes the choice of stored trace (a time-ordered state–action record with a terminal label) harder. An early error changes every later observation, and states that look alike can require different actions because task progress differs, so a trace retrieved by appearance can match the current state in appearance and not in progress, a failure mode reported for closed-loop retrieval [8], [9], [16]. A successful trace is informative only at the task phase the robot has reached, and a failed trace only where its state and progress match the current ones, because its terminal bit does not mark which action was wrong. Retrieval must therefore decide which rollout, which moment within it, and how strongly its action may move the chunk being generated.

Repeated deployment adds a second requirement. A trace store can start from the target-task training traces and grow with the deployed robot’s own successes and failures, which avoids retraining but changes the retrieval distribution after every round, since added traces can displace the neighbors that were useful. The problem is therefore to turn action outcomes labeled by one terminal bit into a correction that applies each trace only at its matching task phase, keeps the action expert’s proposal and weights, and stays bounded as the store grows.

We introduce TraceFlow, a progress-aligned guidance field that turns the action densities of retrieved successful and failed rollouts into a bounded correction to a frozen flow-matching action expert, using one terminal outcome bit per rollout and no other label. A frozen retrieval head maps the VLM state to a retrieval token; the Similarity Gate ranks stored traces by cosine similarity and returns the top- K_+ successful and top- K_- failed candidates; progress alignment selects the action window that follows each matched moment; and a signed term moves the action chunk under integration toward the retrieved successful actions and away from the failed ones. The term is capped relative to the base velocity, and its strength declines to zero before the final solver steps, so the base expert alone produces the final motion. Across rounds only the TraceBank contents change.

We evaluate against the same checkpoint run without guidance; the concurrent methods above are not compared head-to-head (Sec. V). The clearest evidence is on hardware: on an ordered three-fruit packing task the base completes 21 of 50 trials in order with 20 wrong-sequence episodes,

¹The University of Hong Kong, Hong Kong SAR, China.

²Southern University of Science and Technology, Shenzhen, China.

All authors are affiliated with the HKU InfoBodied AI Lab.

*Corresponding author: Yanchao Yang.

TraceFlow 39 of 50 with 2, and one stacking round of the robot’s own rollouts, with no weight update, 47 of 50 with none (Table II). In simulation the gain is selective: with per-suite selected settings, RoboMemArena Sequence rises from 78.92 to 91.50 task success rate (TSR, %) over the hierarchical PrediMem backbone [17] and Transferring from 54.41 to 62.00 at stacking round 2, while Counting and Occlusion fall by 1.12 and 1.42 points and the 26-task aggregate is unchanged (34.92 to 34.99 TSR); on the saturated LIBERO suites and the LIBERO-Plus (Long) OOD variants the changes, +1.45 and +1.27 points ($p = 0.0733$), are within noise (Table I). Our contributions are:

- A guidance field that turns retrieved successful and failed action windows into a capped, early-step correction of a frozen flow-matching action expert from one terminal bit per rollout, with no critic, verifier, dynamics model, or weight update; the cap is what keeps the same retrieval from collapsing the base policy (24.25 versus 57.75 TSR against a 56.75 base, Table III(a)).
- Evidence that the gain is specific to ordering and transfer errors: large on the hardware ordering task and on RoboMemArena Sequence, absent on the memory-dependent Counting and Occlusion suites and on the 26-task aggregate, with the selected and shared configurations named.
- A ten-round stacking study of the robot’s own binary-labeled rollouts: gains peak within a finite number of rounds (Transferring, Joint admission: 54.0 to 62.0 TSR at round 2; LIBERO: 94.8 to 96.8 at round 7), and the best K_+/K_- allocation is not predicted by the bank’s outcome ratio (Table III(d)).

II. RELATED WORK

Action policies. Generalist policies generate actions as tokens, chunks, diffusion samples, or flow-matching solves [1], [4]–[6]. TraceFlow adds no backbone or planner; its single change is a guidance term inside the frozen policy’s solver.

Test-time guidance of generative policies. Diffusion and score models denoise iteratively, whereas flow matching and rectified flow learn a transport field [7], [18]–[20]; classifier and classifier-free guidance change the sampling field [21], [22], and [23] gives the guided field for flow matching, the formal basis of Eq. (4). Diffuser and Diffusion-QL guide sampling for control [24], [25]. For frozen robot policies, DynaGuide steers denoising with a learned dynamics model [14]; Guided Action Flow and test-time Q-guidance differentiate a learned critic trained on rollouts [10], [11]; TACO and RoboMonkey sample chunks and keep the one a learned verifier prefers [12], [13]; CCDP composes conditional diffusion policies so that the failed attempts of the current episode act as negative guidance [15]. TraceFlow needs none of these learned models: its field is a nonparametric mixture over retrieved windows of both outcomes, its negative evidence comes from earlier episodes labeled only by a terminal bit, and its correction is capped relative to the base velocity.

Retrieval and memory at test time. Episodic control and dense dual encoders retrieve pre-encoded records by top- K similarity, and contrastive objectives organize the embedding [26]–[34]; the retrieval head follows these objectives. For VLAs, Retrieve-then-Steer, the closest concurrent method, keeps progress-calibrated successful segments in a long-term memory and injects an aggregated elite prior into an intermediate state of the flow sampler [8]; OptimusVLA replaces the Gaussian initialization with a retrieved task-level prior and adds a learned consistency module [9]; MAP-VLA retrieves learned soft prompts [35]; and a growing demonstration pool extends a frozen policy to new tasks [36]. DynaMem and HALO reuse spatial or visuomotor history [16], [37]. All reuse successes or demonstrations only. TraceFlow retrieves both outcomes, adds a signed score at the current solver state instead of a prior, and stacks the deployed policy’s own binary-labeled rollouts over rounds.

Alternatives that change weights. Reinforcement fine-tuning of flow VLAs improves the expert through a learned critic [38]; hypernetworks, test-time training, TENT, prompt tuning, TTT layers, and RoboTTT change generated weights or an optimized adaptation state [39]–[44]; continual learning addresses forgetting through parameter updates and is evaluated on Continual World and LIBERO [45]–[50]. TraceFlow changes no weight; its rounds change only the TraceBank contents, so Sec. IV measures nonparametric accumulation, not parameter learning or resistance to forgetting.

Failure and progress supervision. REFLECT generates failure summaries, Robot Chain-of-Thought uses subtask rewards, and ProcVLM synthesizes frame-level stage and progress targets [51]–[53]; terminal binary feedback is the sparse alternative [54]. TraceFlow needs only $y \in \{0, 1\}$ per trace, with no failure timestamp, stage label, severity, or corrected segment; it is action-experience reuse, not the memory reasoning that the Counting and Occlusion tasks of RoboMemArena test [17].

III. METHOD

A. Background: How a VLA Policy Produces an Action

A VLA policy maps the current camera image o , a language instruction l , and the robot’s proprioceptive state s (joint or end-effector readings) to an *action chunk* $A \in \mathbb{R}^{H \times d}$: H consecutive robot commands, each a d -dimensional vector of joint or end-effector targets and a gripper command. The policy has two parts. The *VLM* is a large transformer, pretrained on images and text, that encodes (o, l, s) and an optional history h into a vector u that we call the *VLM state*. The *action expert* is a smaller network that turns u into the chunk. With *flow matching*, the expert is a learned *velocity field* $v^{\text{base}}(A, t | u)$: generation starts from a chunk of Gaussian noise A_1 at solver time $t = 1$ and moves it in $1/\Delta t$ small *Euler steps*, $A_{t-\Delta t} = A_t - \Delta t v^{\text{base}}(A_t, t | u)$, until $t = 0$, where A_0 is the chunk the robot executes; each step costs one evaluation of the expert (one NFE). After executing the chunk, the robot observes again and queries the policy again, so a task is a sequence of queries. A *rollout*

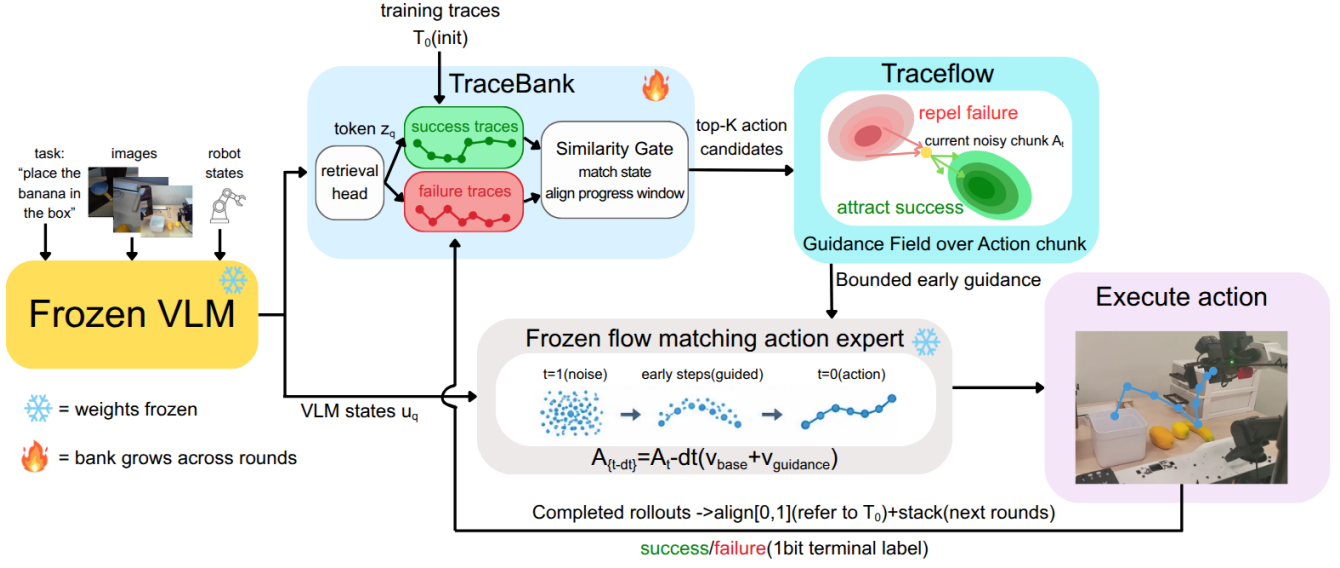


Fig. 1. TraceFlow: observation-to-action guidance and trace reuse. Frozen VLM states u_q condition the action expert and are encoded by a frozen retrieval head into query tokens z_q . Training traces initialize T_0 in TraceBank. The Similarity Gate matches states and aligns progress to return top- K_+ success and optional top- K_- failure action windows. Their densities define a guidance field over action chunks: green attracts the current noisy chunk A_t (yellow) toward successful regions, while red repels it from failed ones. Bounded guidance modifies early flow-integration steps before execution. Completed rollouts receive one terminal success/failure bit and are progress-aligned before stacking for subsequent rounds. Snowflakes indicate frozen weights; the flame indicates bank growth.

(or episode) is one attempt at a task from start to success, failure, or timeout; its *terminal bit* $y \in \{0, 1\}$ records the outcome. A *frozen* policy is one whose weights are all fixed for the whole deployment, the setting of this paper.

TraceFlow leaves this generation path unchanged and adds an external *experience path* with four modules (Fig. 1; Secs. III-B–III-E): a TraceBank that stores completed rollouts as traces with their terminal bit; a retrieval step that selects stored actions comparable to the current state and task progress; a guidance term that moves the chunk under generation toward retrieved successful actions and away from retrieved failed ones, with bounded strength and only in the early Euler steps; and an admission step that appends new rollouts for later rounds. The VLM, the action expert, and the retrieval head stay fixed throughout.

B. Frozen Policy and TraceBank Interface

A *trace* is the record of one rollout: its *retrieval keys* $\mathcal{Z}_i$ (one vector per time step, defined below), its executed actions $\mathcal{A}_i$, task and episode-boundary metadata m_i , and its terminal bit y_i . The training rollouts of the task initialize the bank:

$$\tau_i = (\mathcal{Z}_i, \mathcal{A}_i, m_i, y_i), \quad T_0 = \{\tau_i : i \in \mathcal{I}_{\text{train}}\}, \quad (1)$$

where $\mathcal{I}_{\text{train}}$ indexes the training rollouts and T_r denotes the bank after r deployment rounds. A retrieval key (or *token*) is a fixed-length unit vector computed from a VLM state by the *retrieval head*, a small three-layer network trained offline with a contrastive objective so that states from similar task situations receive similar tokens; similarity between two tokens is their *cosine similarity*, the dot product of the two unit vectors (1 for identical directions, 0 for unrelated ones). An *anchor* is a stored token at one time step of a trace

together with the actions that follow it, and an *action window* is the H consecutive stored actions starting at an anchor, so that a window has the same shape as a chunk. At query q the VLM encodes the inputs, and the same state serves both paths:

$$u_q = \text{VLM}_{\theta}(o_q, l, s_q, h_q), \quad \mathcal{R}_q^{\pm} = \text{Retrieve}_{\pm}(u_q, T_r). \quad (2)$$

Here $\text{Retrieve}_{\pm}$ applies the retrieval head to u_q to obtain the query token z_q and then the Similarity Gate of Sec. III-C, returning a set $\mathcal{R}_q^+$ of successful and a set $\mathcal{R}_q^-$ of failed action windows; in parallel, u_q conditions the action expert as in the unmodified policy. Sec. IV lists the head variants used for each backbone.

C. Similarity Gate and Progress-Aligned Retrieval

The Similarity Gate computes the cosine similarity between z_q and every stored anchor and ranks anchors separately by outcome. From the success bank it keeps the best anchor of each trace before selecting the K_+ highest, so that no single trace fills the set; from the failure bank it selects the K_- highest anchors, which may come from the same trace, because a failed rollout is negative evidence only near the states where it was recorded. Ranking selects *which rollout*; *progress alignment* selects *which moment*: starting at the matched anchor, a forward-only tracker advances along the stored trace as actions are executed and corrects its position by local state matching, without crossing the episode boundary, so that the returned window belongs to the task phase the robot has reached. No progress or failure-location label is used. The returned action windows, not their tokens, form the guidance field.

D. TraceFlow Action Guidance

Retrieved windows guide, not replace, the noisy chunk $A_t \in \mathbb{R}^{H \times d}$ in the policy’s normalized action coordinates. For each bank $y \in \{+, -\}$, every retrieved window A_i^y receives a weight $w_i^y = \text{softmax}_i(10a_i^y)$, where a_i^y is the cosine similarity between its anchor and the query (for the failure bank all a_i^- are set to one). The windows define a *kernel density*: each window is the center of a Gaussian of width σ_y in chunk space, and the weighted sum of these Gaussians is a density p_y whose value is high near retrieved windows of outcome y . Its *score*, the gradient of the log-density with respect to the chunk, points from A_t toward the nearby windows:

$$p_y(A | z_q) = \sum_i w_i^y \mathcal{N}(A; A_i^y, \sigma_y^2 I),$$

$$s_t^y = \nabla_{A_t} \log p_y(A_t | z_q) = \frac{\sum_i r_{i,t}^y A_i^y - A_t}{\sigma_y^2}, \quad (3)$$

where $r_{i,t}^y \propto w_i^y \mathcal{N}(A_t; A_i^y, \sigma_y^2 I)$, normalized over i . Within the retrieved set, success responsibilities combine retrieval similarity with action proximity; failure responsibilities use uniform prior weights and depend on action proximity. The score is therefore recomputed as A_t moves rather than fixed to one average of the windows. Following density-based guidance for flow matching [23], the success score is added as attraction and the failure score as repulsion:

$$v_{q,t}^{\text{guide}} = \text{Bound}(-\lambda(t)s_t^+, \lambda(t)\beta_- s_t^-; v_{q,t}^{\text{base}}),$$

$$A_{t-\Delta t} = A_t - \Delta t (v_{q,t}^{\text{base}} + v_{q,t}^{\text{guide}}). \quad (4)$$

Here $v_{q,t}^{\text{base}}$ is the expert’s own velocity at this step, $\lambda(t) \geq 0$ is the guidance strength, β_- weights the failure term, and the minus sign on s_t^+ is what moves A_t up the success density under the backward integration of Eq. (4). Bound is the *cap*: it rescales the success term to a norm of at most $c_+ \|v^{\text{base}}\|$, the failure term to at most $c_- \|v^{\text{base}}\|$, and their sum to at most $c \|v^{\text{base}}\|$, keeping each direction unchanged; c_+, c_-, c are dimensionless. Without the cap (*direct* guidance), the norm of s_t^y grows with the distance between A_t and the windows and is not tied to the base velocity, so the guidance can dominate the base flow; *bounded* guidance cannot exceed a fixed fraction of it (Table III(a)). A missing bank contributes zero. The strength $\lambda(t)$ declines linearly from λ_{max} at $t = 1$ to zero at t_{cut} and stays zero for the remaining steps, which refine contact and placement with the base expert alone. Retrieval is done once per query; the scores are recomputed at every active step.

E. Trace Stacking and Progress Normalization

After a chunk is executed, the next observation starts a new query. A *round* collects a batch of completed rollouts before the bank is updated; *stacking* appends their traces to the bank without changing any weight. The *admission rule* decides which outcomes enter: Success-only admits new successes, Failure-only new failures, and Joint both; existing traces stay retrievable in every case. A trace becomes available in the

next round only, so a rollout never retrieves its own future. In this paper, continual learning means this repeated trace accumulation, not parameter training.

Two real executions of the same task pause and move at different speeds, so the frame index of a trace is not a measure of task progress. Before a hardware trace τ is admitted, we express it on the normalized progress clock $[0, 1]$ of the training traces and append it to the bank of its outcome y_τ :

$$\tilde{\tau} = \text{Align}_{[0,1]}(\tau; \mathcal{T}_0), \quad \mathcal{T}_{r+1}^y = \mathcal{T}_r^y \cup \{\tilde{\tau} \mid y_\tau = y\}. \quad (5)$$

$\text{Align}_{[0,1]}$ assigns each state of τ the progress value of the training states it matches, not its elapsed time: for every state it takes the similarity-weighted median [55] of the progress values of the most similar same-task anchors (an optional runtime estimate is clipped near this vote and blended in), and for successful traces it projects the sequence onto a non-decreasing curve on $[0, 1]$ by pool-adjacent-violators regression [56] and limits each forward increment to $F_{\text{max}} / \max(L_{\text{ref}} - 1, 1)$, where L_{ref} is the reference trace length and F_{max} the allowed advance in reference frames. Successful runs of different lengths thus cover the same start-to-completion range, while a failed run keeps its local anchor match and its estimated endpoint instead of being stretched to completion. Optional gripper-event landmarks (median normalized event times of the training traces) keep the order of open and close events; after removing holds, the actions are resampled on the new clock. These steps change only the progress clock, not the outcome label or any weight.

IV. EXPERIMENTS

We answer three questions on simulation and real-robot tasks. RQ1: where does retrieved guidance help, and which integration rule keeps the base flow intact? RQ2: does admitting binary-labeled rollouts to TraceBank raise success repeatably, and for how many rounds? RQ3: does the bank’s success-to-failure ratio predict the signed retrieval allocation K_+/K_- ? Tables I–II report task success, Fig. 3 the stacking rounds, Table III the integration and retrieval controls, and Table IV the ablations.

A. Benchmarks and Protocol

Simulation. We use $\pi_{0.5}$ [6] as the reference VLA and PrediMem [17], RoboMemArena’s hierarchical model in which an Upper VLM selects subtasks and a Lower action policy executes them, to test whether guidance also benefits a dual-tower policy. Both use flow-matching action experts; all policy and retrieval weights are fixed after task-specific setup. Every comparison is against the same checkpoint run without guidance; the concurrent test-time methods of Sec. II are not run. We evaluate on the four LIBERO suites [50], on 2,519 paired OOD task variants of LIBERO-Plus (Long) [57] (the released `libero_10` suite, which varies texture, viewpoint, language, lighting, layout, initial state, and sensor noise while TraceBank holds only the original LIBERO-10 traces), and on all 26 long-horizon RoboMemArena tasks [17] (Arena in the tables). LIBERO is a saturation check.

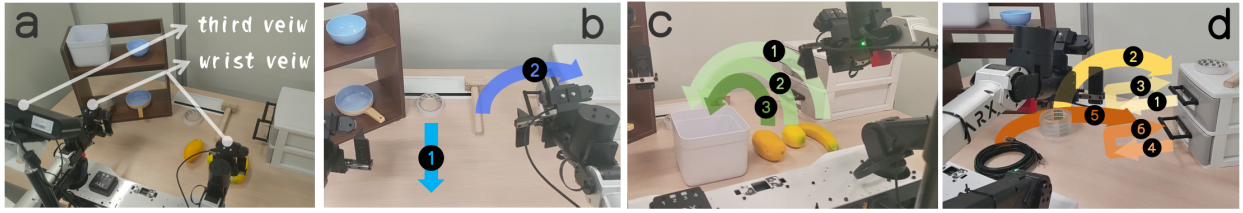


Fig. 2. Real-world task suite in a cluttered storage-room scene. (a) The dual-arm platform with a third-person camera (Astra Pro Plus) and two wrist cameras (Gemini Pro). (b) T1, two-step relocation: move the double-sided tape down (1), then place the wooden hammer on the white cabinet (2). (c) T2, ordered packing of three fruits into the white box along the numbered paths. (d) T3, two ordered three-step branches across similar drawers: open the upper drawer, place the single-sided tape, close it (1-3); open the lower drawer, place the cable, close it (4-6). Arrow colors separate objects or branches; the numbers give the required order.

TABLE I
Simulation results.

Evaluation	Backbone	Performance		Δ
		Base	TraceFlow	
LIBERO four-suite	$\pi_{0.5}$	96.85	98.30	+1.45
LIBERO-Plus (Long)	$\pi_{0.5}$	79.83	81.10	+1.27
Arena Seq.+Trans.	$\pi_{0.5}$	56.75/62.92	57.75/63.42	+1.00/+0.50
Arena Sequence	PrediMem	78.92/83.09	91.50/95.58	+12.58/+12.49
Arena Transferring	PrediMem	54.41/66.34	62.00/69.08	+7.59/+2.74
Arena Counting	PrediMem	26.61/56.12	25.49/53.50	-1.12/-2.62
Arena Occlusion	PrediMem	17.11/42.34	15.69/40.60	-1.42/-1.74
Arena Full26	PrediMem	34.92/56.01	34.99/55.04	+0.07/-0.97

SR is success rate; TSR/CSR are task/completion success rates (%). Seq./Trans. abbreviate Sequence/Transferring. PrediMem is the hierarchical backbone; Upper denotes its subtask-selector features. Dir./Bnd. mean uncapped/bounded guidance; Joint admits both successful and failed new traces. Bold denotes the better value within each comparison. Base and TraceFlow use the same checkpoint and evaluator within each row. These are selected settings, not one shared configuration or exclusively pre-stacking results: 98.30 averages LIBERO suite-wise selections (one shared configuration gives 98.00); Sequence uses Upper-Dir., $K_+ = 50$; Transferring is Joint R2, 8/8. Full26 is a separate suite-conditioned aggregate, not the average of the selected suite rows. Counting/Occlusion rows use the suite-adapted settings of Table IV(b). PrediMem rows use its May checkpoint [17] with the July benchmark; the $\pi_{0.5}$ Arena row is an eight-task, 400-episode probe. LIBERO-Plus (Long) uses $n = 2,519$ paired trials ($p = 0.0733$).

Every Arena number is re-measured in our testbed with the official simulation and benchmark settings and the latest official checkpoints at the time of the run, not imported from the RoboMemArena paper, because published numbers differ across benchmark versions. TSR counts episodes that complete every subtask in order; CSR counts stages credited by the official task scorer, including its sequential-stage checks. Retrieval heads: for $\pi_{0.5}$, the head takes one 2048-D VLM state (S) or three states at frame offsets $[-20, -10, 0]$ (T) through a 1024-D hidden layer; D5/D10 sample bank anchors every five/ten frames (1024/2048-D tokens), and T-Dense5 combines T input with D5 sampling. For PrediMem, the head reads the subtask-selector state (Upper), the action-policy state (Lower), or both with Upper-state age indicators (Fusion), and always guides the Lower expert (Table IV(a1)). The Full26 comparison uses seeds 50-100, 51 trials per task, and 1,326 episodes per row; the $\pi_{0.5}$ Arena probe uses eight tasks and 400 episodes. The reference bounded configuration uses 10 Euler steps ($\Delta t = 0.1$), each with one base-velocity function evaluation (NFE), $K_+/K_- = 16/8$, $\sigma_+ = \sigma_- = 0.30$, $\lambda_{\max} = 0.20$, $\beta_- = 0.10$, $t_{\text{cut}} = 0.30$, and $(c_+, c_-, c) = (0.20, 0.10, 0.20)$ (Sec. III-D). Changes are

TABLE II
Real-world results.

Task	Method	TSR $\uparrow$	WS $\downarrow$	CSR $\uparrow$
T1: tape + hammer (2)	Base	8/50	0/50	34/100
	TraceFlow	16/50	0/50	52/100
T2: ordered fruits (3)	Base	21/50	20/50	123/150
	TraceFlow	39/50	2/50	129/150
	TraceBank-Stack	47/50	0/50	145/150
T3: tape + cable (6)	Base	0/10	8/10	8/60
	TraceFlow	1/10	0/10	31/60

Entries count trials (TSR/WS) or annotated subtasks (CSR); bold marks the preferable value per task. TraceBank-Stack is one additional T2 round using joint guidance from stacked success/failure traces normalized by Eq. (5) in Sec. III-E, without a new policy or weight update.

percentage points; paired outcomes use exact McNemar tests only when episodes are paired. Unless a mean is specified, each cell is one evaluation pass over the stated trials, not a multi-seed mean. Table I's PrediMem Sequence/Transferring selections use 200 trials versus 204 base trials; Full26 and Counting/Occlusion use 51 trials per task. These selected summaries do not estimate uncertainty over configuration selection.

Real-world. Figure 2 shows the three cluttered-room tasks: tape-and-hammer relocation (T1), ordered three-fruit packing (T2), and a six-stage tape and cable task across similar drawers (T3). WS counts wrong-sequence episodes (lower is better). Hardware TSR requires all stages in order; hardware CSR, unlike Arena's sequential scorer, credits subtasks irrespective of order. We use an ARX AC-One dual-arm robot, removing its stock teleoperation fixtures and collecting demonstrations with Meta Quest 3S controllers. The frozen $\pi_{0.5}$ baseline receives action-expert-only fine-tuning for 30k steps at batch size 64 with other OpenPI defaults. TraceFlow uses the same checkpoint, a T-Dense5 head trained for 30 epochs (three causal anchors; five-frame bank sampling), $K_+/K_- = 8/8$, and total cap $c = 0.10$; all other guidance settings match this reference bounded configuration.

B. Answering the Research Questions

1) *RQ1: Retrieved Guidance Helps Selectively; Bounds Stabilize It: Real-world.* Table II shows three task-specific effects. On T1, TraceFlow raises TSR from 8/50 to 16/50 and

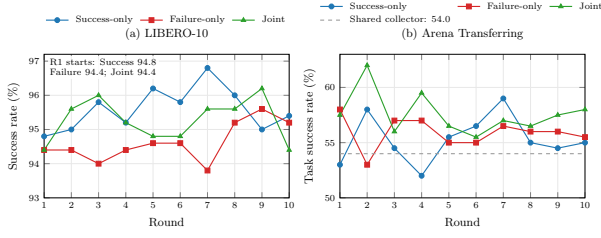


Fig. 3. Ten-round trace stacking. Success-only (blue), Failure-only (red), and Joint (green) append new successes, failures, and both, respectively; previously stored traces stay queryable. Each round collects 500/200 rollouts in LIBERO/Arena. LIBERO Success-only peaks at R7 (96.8% SR; R1: 94.8); Failure-only/Joint peak at R9 (95.6/96.2) and end at 95.2/94.4. Arena Success-only, Failure-only, and Joint peak at R7, R1, and R2 (59.0%, 58.0%, and 62.0% TSR), respectively; Failure-only ends at 55.50/64.50 TSR/CSR. The dashed line is the shared Arena collector (54.0%), the policy that generated the rollouts before stacking; R1 starts are LIBERO’s empty-bank scores. R denotes the collection round.

CSR from 34/100 to 52/100; the base failed mostly at the tape pickup (observed, not counted separately). On T2, the base’s wrong-sequence episodes mostly swap stages 2 and 3; TraceFlow raises first-round TSR from 21/50 to 39/50 and lowers WS from 20/50 to 2/50. On T3, the upper and lower drawers look alike and require similar motions, so the base usually opens the lower drawer first and then stalls or moves unsafely from that state; TraceFlow records 0/10 WS, raises CSR from 8/60 to 31/60, and completes one of ten six-subtask episodes. With ten T3 trials this result is descriptive.

Simulation. On the eight-task $\pi_{0.5}$ probe of Sequence and Transferring (Table III(a); $K_+ = 8$, $n = 400$), the same retrieval gives 24.25/41.62 TSR/CSR with the uncapped term and 57.75/63.42 with the cap, against a 56.75/62.92 base; the capped result is 1.00/0.50 points above the base. The three-seed LIBERO-10 probe in Table III(a) gives 95.37/95.07% direct/bounded mean SR ($p = 0.580$), without a detected paired difference. In Table I, the gains concentrate on PrediMem Sequence (78.92 to 91.50 TSR, Upper-Dir., $K_+ = 50$) and Transferring (54.41 to 62.00, Joint stacking at R2); under the shared configuration the 26-task aggregate changes by +0.07/−0.97 TSR/CSR, and Counting and Occlusion fall 1.12/2.62 and 1.42/1.74 below their bases, so the simulation gain is specific to the ordering and transfer suites. On LIBERO the four-suite success rate rises from 96.85 to 98.30, and on LIBERO-Plus (Long) from 79.83 to 81.10 ($n = 2,519$ paired trials, $p = 0.0733$).

2) *RQ2: Binary-Labeled Stacking Improves Briefly: Simulation.* The three branches in Fig. 3 differ only in the admission rule: Success-only appends new successes, Failure-only new failures, and Joint both; earlier traces stay retrievable. Every Arena branch keeps a fixed eight-task bank of 800 successful demonstrations from four Sequence and four Transferring tasks. The LIBERO self-collection study instead begins R1 with empty banks. Each branch is one ten-round history with seed 7, not independently replicated histories. Admission raises success for a finite number of rounds: LIBERO Success-only rises from 94.8% at R1 to its 96.8% peak at R7, and Arena Joint from the 54.0% collector to

TABLE III
Guidance and TraceBank studies.

(a) Integration rule, $\pi_{0.5}$ (Arena: Seq.+Trans., $K_+ = 8$, $n = 400$)				
	Base	Direct	Bounded	
Arena TSR/CSR	56.75/62.92	24.25/41.62	57.75/63.42	
L10 mean SR	94.60	95.37	<u>95.07</u>	
(b) Retrieval breadth K_+ (26 trials/task)			(c) TraceBank scope (Seq.; Dir.)	
K_+	Seq. (Dir.)	Trans. (Bnd.)	Scope Bank	n Seq.
8	90.38/94.23	57.69/68.11	Single Seq.	200 83.50/86.58
16	92.31/95.35	50.96/62.50	Two Seq.+Trans.	200 90.50/92.33
32	93.27/95.03	54.81/64.10	Four Full26	204 80.39/89.13
50	93.27/95.51	57.69/66.03		
100	89.42/92.15	54.81/63.14		
(d) Admission rule $\times K_+/K_-$ allocation (Trans.; Bnd.; R1–R10)				
Admission	K_+/K_-	Mean TSR/CSR	Peak TSR/CSR (round)	
Joint	8/8	57.60/65.92	62.00/69.08 (R2)	
Joint	16/8	56.10/64.88	58.00/65.25 (R9)	
Joint	8/16	<u>56.30/65.03</u>	59.00/66.75 (R1)	
Success	8/8	55.30/64.62	59.00/66.75 (R7)	
Success	16/8	56.30/65.47	59.50/67.75 (R10)	
Success	8/16	<u>56.30/65.04</u>	59.50/66.83 (R7)	
Failure	8/8	<u>55.90/64.85</u>	58.00/67.17 (R1)	
Failure	16/8	57.10/65.34	60.50/67.33 (R8)	
Failure	8/16	55.80/64.84	<u>60.50/67.67</u> (R9)	

TSR/CSR (%) except L10 (LIBERO-10) mean SR; bold/underline mark best/second-best. (b) retrieves from the success bank only. L10 uses seeds 7/17/27 with 1,000 episodes each: Base 93.5/94.1/96.2, Dir. 95.6/94.9/95.6, Bnd. 95.6/94.3/95.3; the pooled Dir.–Bnd. McNemar test gives $p = 0.580$. Seq./Trans.: Sequence/Transferring; Dir./Bnd.: direct (uncapped)/bounded ($c = 0.20$); Table IV shares these conventions. Backbones: (a) $\pi_{0.5}$; (b)–(d) PrediMem. In (c), Full26 covers all four suites, so bank content changes with bank size. In (d), means are ranked within admission groups and peaks across all nine settings; all allocations re-evaluate the three 8/8 branches of Fig. 3(b) without new collection. Success/Failure abbreviate Success-only/Failure-only admission. Mean averages R1–R10; Peak reports the highest-TSR round and its accompanying CSR, not separately maximized metrics.

62.0% at R2; every branch is below its peak at R10 (Fig. 3). Arena Failure-only peaks at R1 (58.00/67.17 TSR/CSR) and ends at 55.50/64.50 in R10 (Fig. 3(b)), above the collector and below Joint at both the peak and R10. R7 and R2 are observed peaks of single histories, not stopping rules. Table III(d) re-evaluates the three 8/8 branches under 16/8 and 8/16 without recollecting banks.

Real-world. On T2, TraceBank-Stack admits the first round’s successful and failed rollouts, normalized by Eq. (5), and applies joint guidance with no weight update; the stacked round reaches 47/50 TSR, 145/150 CSR, and 0/50 WS (Table II). Progress normalization biases the retrieved windows toward the actions stored at the matched phase but does not replay a trajectory: the frozen VLA is re-queried at every observation and the guidance stays bounded. In hardware rollouts we observed, without counting them, recoveries from local deviations back to the intended sequence.

3) *RQ3: Sweeps Establish No Reliable Bank-Ratio Allocation Rule: Simulation.* Table III(b)–(d) varies retrieval breadth, bank scope, and signed allocation. Sequence peaks at $K_+ = 32$ –50, Transferring at $K_+ = 8$ by CSR; both decline at 100 (b). The two-suite bank leads on Sequence (c), but content and size change together. Joint has its

TABLE IV
Retrieval and failure ablations.

(a1) Head source $\times$ integration (8-task bank; $K_+/K_- = 16/0$; $n = 400$)			(a2) S/T = 1/3 input anchors (8-task agg.; $K_+ = 8$; $n = 400$)		
Head, rule	Seq.	Trans.	Setting	S	T
Lower, Dir.	73.50/83.67	26.50/36.17			
Upper, Dir.	90.50/92.33	50.50/60.17	D10, Dir.	33.75/47.83	24.25/39.25
Fusion, Dir.	84.50/90.67	49.00/62.58	D10, Bnd.	55.00/62.33	54.25/61.00
Lower, Bnd.	77.50/84.75	55.50/65.58	D5, Bnd.	55.50/63.17	57.75/63.42
Upper, Bnd.	83.50/91.08	54.00/62.50			
Fusion, Bnd.	78.00/87.33	58.00/68.50			

(b) Counting/Occlusion audit: integration $\times$ bank scope (TSR/CSR)					
Setting	Integration	Bank	n /task	Counting	Occlusion
Frozen base	none	none	51	26.61/56.12	17.11/42.34
Full26	Dir.	Full26	51	15.97/37.16	8.38/33.69
Fixed cap	Bnd. (.20)	Full26	51	23.53/48.83	13.90/39.50
Adaptive	Bnd. (fitted)	Full26	10	21.43/49.05	11.82/36.10
Suite adaptive	Bnd. (.01/.04)	suite-only	51	25.49/53.50	15.69/40.60

(c) Persistent Upper-error audit (Full26; Bnd.; guidance minus base)			
Suite	Metric	Δ (pp)	p
Counting	Pre-error progress	-4.67	.049
Counting	Post-error completion	-7.47	.037
Occlusion	Pre-error progress	+0.15	.578
Occlusion	Post-error completion	-9.11	9.34×10^{-5}

Blocks (a1)/(b)/(c) use PrediMem; (a2) uses $\pi_{0.5}$. Upper/Lower denote subtask-selector/action-policy feature sources; Fusion combines both. The 8-task bank contains 800 successful demonstrations from four Seq. and four Trans. tasks. D5/D10 mean five-/ten-frame anchor sampling (1024-/2048-D output tokens). S/T in (a2) denote single-/three-anchor inputs, not task suites; (a1) reports Seq. and Trans. separately and (a2) their eight-task aggregate. In (b), caps are given per row; the 10-trial Adaptive row is an unranked matched-seed probe. Suite-only restricts the bank to the evaluated suite; fitted denotes the adaptive cap probe, not a fixed cap. Suite adaptation changes cap and bank scope together. Cells are single passes over the stated trials; (a1)/(a2) use 400 episodes per setting. Block (c) uses bounded guidance, Full26 T-Dense5 Fusion, $K_+ = 16$, and 51 trials/task; the boundary is the retrospectively identified start of the persistent wrong-Upper suffix. Pre-error progress: fraction of all stages completed before it; post-error completion: fraction of remaining stages completed in episode pairs with the same boundary goal rank (different denominators, not task SR). p from an exact sign test (pre) and exact McNemar (post). A diagnostic, not causal evidence.

highest mean CSR at 8/8, Success and Failure at 16/8; within-branch mean TSR varies by at most 1.5 points (d). Comparing admission branches, not measured bank ratios against optimal allocations, establishes no reliable ratio rule without excluding such a relationship. Hardware budgets remain empirical.

C. Ablation Studies

Table IV reports the Arena ablations. In (a1), Sequence is best with the Upper source and direct integration and Transferring with the Fusion source and bounded integration, so the source-integration preference is task-dependent. In (a2), three-anchor input (T) lowers the Dense10 head (D10: anchors every ten frames) but raises bounded Dense5 (D5: every five frames) from 55.50/63.17 to 57.75/63.42 TSR/CSR on the eight-task aggregate, which is why T-Dense5 is the unified head; we claim no general advantage of temporal context. D5/D10 also use 1024/2048-D output tokens, respectively, so only S versus T within one density fixes both sampling and output dimension (Sec. III-C).

In (b), the cap and the suite-restricted bank reduce the Counting and Occlusion deficits (from 15.97/37.16 and 8.38/33.69 with the uncapped Full26 bank to 25.49/53.50 and 15.69/40.60 with suite adaptation), but no setting reaches the base (26.61/56.12 and 17.11/42.34). Suite adaptation changes the cap and the bank scope together, so it does not isolate the effect of a weaker guidance. Table IV(c) examines whether the deficits arise only after persistent Upper errors. Occlusion changes by +0.15 pp in pre-error progress and -9.11 pp in post-error completion; Counting already loses 4.67 pp in pre-error progress. Thus Upper error alone does not explain both suites; this retrospective association supplies no causal explanation or online switch criterion. These deficits remain unresolved.

V. LIMITATIONS AND CONCLUSION

Four limits bound the claims. First, the guidance moves the action chunk toward stored actions and is not designed to reconstruct missing upstream event information: Counting and Occlusion stay below the base under every tested cap and bank scope, and the audit of Table IV(c) associates part of the Occlusion loss with persistent subtask-selection errors without establishing a cause. Second, the simulation headline cells are per-suite selected configurations evaluated in one pass, the 26-task aggregate under the shared configuration is unchanged, and the LIBERO and LIBERO-Plus changes are within noise; the T3 result rests on ten trials. Third, every admitted trace changes which anchors rank highest, so stacking gains are finite and branch-specific (each branch peaks between R1 and R9 and is below its peak at R10, Fig. 3), the ten-round histories are single runs, and the sweeps establish no bank-ratio rule for K_+/K_- (Table III(d)). Fourth, the concurrent test-time methods of Sec. II are not compared head-to-head; the only baseline is the same checkpoint without guidance.

TraceFlow adds a bounded, progress-aligned success and failure field to a frozen flow-matching action expert, using one terminal bit per rollout and no weight update. Its effect is specific: it removes ordering errors on a real robot (21 to 39 of 50 ordered completions, 47 after one stacking round) and raises RoboMemArena Sequence from 78.92 to 91.50 TSR, and it does not help where a task fails for lack of event information in the policy state.

REFERENCES

- [1] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183.
- [2] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2679–2713.
- [3] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems*, 2024.
- [4] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, 2024.
- [5] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, 2025.

- [6] K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 17–40.
- [7] Y. Lipman *et al.*, “Flow matching for generative modeling,” in *International Conference on Learning Representations*, 2023.
- [8] J. Zhao *et al.*, “Retrieve-then-steer: Online success memory for test-time adaptation of generative VLAs,” *arXiv preprint arXiv:2605.10094*, 2026.
- [9] Z. Li *et al.*, “Global prior meets local consistency: Dual-memory augmented vision-language-action model for efficient robotic manipulation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, arXiv:2602.20200.
- [10] L. Yang *et al.*, “Guided action flow: Q-guided inference for flow-matching vision-language-action policies,” *arXiv preprint arXiv:2607.02092*, 2026.
- [11] Z. Zhou *et al.*, “Test-time gradient guidance of flow policies in reinforcement learning,” *arXiv preprint arXiv:2606.11087*, 2026.
- [12] S. Yang *et al.*, “Steering vision-language-action models as anti-exploration: A test-time scaling approach,” *arXiv preprint arXiv:2512.02834*, 2025.
- [13] J. Kwok *et al.*, “RoboMonkey: Scaling test-time sampling and verification for vision-language-action models,” *arXiv preprint arXiv:2506.17811*, 2025.
- [14] M. Du *et al.*, “DynaGuide: Steering diffusion policies with active dynamic guidance,” in *Advances in Neural Information Processing Systems*, 2025, arXiv:2506.13922.
- [15] A. Razmjoo *et al.*, “CCDP: Composition of conditional diffusion policies with guided sampling,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2025, pp. 20036–20043.
- [16] R. Shah *et al.*, “Memory retrieval in visuomotor policies for long-horizon robot control,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, 2026.
- [17] H. Lei *et al.*, “RoboMemArena: A comprehensive and challenging robotic memory benchmark,” *arXiv preprint arXiv:2605.10921*, 2026.
- [18] J. Ho *et al.*, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, 2020.
- [19] Y. Song *et al.*, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations*, 2021.
- [20] X. Liu *et al.*, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *International Conference on Learning Representations*, 2023.
- [21] P. Dhariwal *et al.*, “Diffusion models beat GANs on image synthesis,” in *Advances in Neural Information Processing Systems*, 2021.
- [22] J. Ho *et al.*, “Classifier-free diffusion guidance,” in *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.
- [23] R. Feng *et al.*, “On the guidance of flow matching,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 16993–17029.
- [24] M. Janner *et al.*, “Planning with diffusion for flexible behavior synthesis,” in *International Conference on Machine Learning*, 2022, pp. 9902–9915.
- [25] Z. Wang *et al.*, “Diffusion policies as an expressive policy class for offline reinforcement learning,” in *International Conference on Learning Representations*, 2023.
- [26] C. Blundell *et al.*, “Model-free episodic control,” in *International Conference on Learning Representations*, 2016.
- [27] A. Pritzel *et al.*, “Neural episodic control,” in *International Conference on Machine Learning*, 2017, pp. 2827–2836.
- [28] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.
- [29] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [30] A. van den Oord *et al.*, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [31] T. Chen *et al.*, “A simple framework for contrastive learning of visual representations,” in *International Conference on Machine Learning*, 2020.
- [32] K. He *et al.*, “Momentum contrast for unsupervised visual representation learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 9729–9738.
- [33] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning*, 2021, pp. 8748–8763.
- [34] P. Khosla *et al.*, “Supervised contrastive learning,” in *Advances in Neural Information Processing Systems*, 2020.
- [35] R. Li *et al.*, “MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation,” in *IEEE International Conference on Robotics and Automation*, 2026.
- [36] J. Park *et al.*, “Retrieve, don’t retrain: Extending vision language action models to new tasks at test time,” *arXiv preprint arXiv:2606.15631*, 2026.
- [37] P. Liu *et al.*, “DynaMem: Online dynamic spatio-semantic memory for open world mobile manipulation,” in *IEEE International Conference on Robotics and Automation*. IEEE, 2025, pp. 13346–13355.
- [38] Z. Wang *et al.*, “Q-VGM: Q-guided value-gradient matching for flow-matching VLA policies,” *arXiv preprint arXiv:2606.08015*, 2026.
- [39] D. Ha *et al.*, “Hypernetworks,” in *International Conference on Learning Representations*, 2017.
- [40] Y. Sun *et al.*, “Test-time training with self-supervision for generalization under distribution shifts,” in *International Conference on Machine Learning*, 2020, pp. 9229–9248.
- [41] D. Wang *et al.*, “Tent: Fully test-time adaptation by entropy minimization,” in *International Conference on Learning Representations*, 2021.
- [42] M. Shu *et al.*, “Test-time prompt tuning for zero-shot generalization in vision-language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [43] Y. Sun *et al.*, “Learning to (learn at test time): RNNs with expressive hidden states,” in *International Conference on Machine Learning*, 2025, pp. 57503–57522.
- [44] Y. Jiang *et al.*, “RoboTTT: Context scaling for robot policies,” *arXiv preprint arXiv:2607.15275*, 2026.
- [45] J. Kirkpatrick *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [46] D. Lopez-Paz *et al.*, “Gradient episodic memory for continual learning,” in *Advances in Neural Information Processing Systems*, 2017.
- [47] D. Rolnick *et al.*, “Experience replay for continual learning,” in *Advances in Neural Information Processing Systems*, 2019.
- [48] P. Buzzega *et al.*, “Dark experience for general continual learning: A strong, simple baseline,” in *Advances in Neural Information Processing Systems*, 2020.
- [49] M. Wolczyk *et al.*, “Continual world: A robotic benchmark for continual reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2021.
- [50] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for life-long robot learning,” in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.
- [51] Z. Liu *et al.*, “REFLECT: Summarizing robot experiences for failure explanation and correction,” in *Conference on Robot Learning*, 2023, pp. 3468–3484.
- [52] K. Zhang *et al.*, “Learning manipulation skills through robot chain-of-thought with sparse failure guidance,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2025, pp. 8012–8018.
- [53] Y. Feng *et al.*, “ProcVLM: Learning procedure-grounded progress rewards for robotic manipulation,” *arXiv preprint arXiv:2605.08774*, 2026.
- [54] J. Liang *et al.*, “Self-supervised learning of long-horizon manipulation tasks with finite-state task machines,” in *Learning for Dynamics and Control Conference*, 2021, pp. 484–497.
- [55] F. Y. Edgeworth, “On a new method of reducing observations relating to several quantities,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 25, no. 154, pp. 184–191, 1888.
- [56] M. J. Best *et al.*, “Active set algorithms for isotonic regression: A unifying framework,” *Mathematical Programming*, vol. 47, pp. 425–439, 1990.
- [57] S. Fei *et al.*, “LIBERO-Plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.